

Сесія. Документація script.js

1. Зчитування даних з пам'яті

```
JS script.js > ...
1  //1. Зчитування даних з пам'яті
2  let cart = JSON.parse(localStorage.getItem('cart')) || [];
3  const user = JSON.parse(localStorage.getItem('userProfile'));
4  const greetingEl = document.getElementById('user-greeting');
```

let cart = ...: Створює змінну cart (кошик), яку можна змінювати.

localStorage.getItem('cart') — шукає в пам'яті браузера збережені дані з назвою 'cart'. Оскільки там лежить текст, метод **JSON.parse(...)** перетворює цей текст назад у JavaScript-масив.

|| [] — це запасний варіант (оператор АБО). Якщо в пам'яті нічого немає (користувач зайшов вперше), cart стане порожнім масивом [], щоб код не видав помилку далі.

const user = ...: Робить те саме для профілю користувача. Шукає **'userProfile'**, перетворює з тексту в об'єкт і записує у сталу змінну user.

const greetingEl = ...: Шукає у твоєму HTML-файлі елемент, який має **id="user-greeting"** (блок для привітання), і зберігає посилання на нього в змінну **greetingEl**.

2. Відображення привітання

```

5
6 //2. Вибображення привітання
7 if (greetingEl && user && user.username) {
8   greetingEl.innerHTML = `Вітаємо, ${user.username}! <button onclick="logoutUser()" style="margin-left: 1rem; padding: 0.3rem;`
9 }
0

```

if (...): Перевіряє одразу три умови: чи існує на сторінці блок привітання (**greetingEl**), чи знайдено дані користувача (**user**), і чи є в цих даних ім'я (**user.username**).

greetingEl.innerHTML = ...: Якщо всі три умови виконані, цей рядок вставляє всередину блоку привітання HTML-код: текст "Вітаємо, [Ім'я]!" та кнопку "Вийти". При натисканні на кнопку (onclick) запуститься функція **logoutUser()**.

3. Функція виходу з акаунту

```

0
1 //3. Функція виходу з акаунту
2 function logoutUser() {
3   localStorage.removeItem('userProfile');
4   location.reload();
5 }

```

function logoutUser() {: Оголошує функцію виходу.

localStorage.removeItem('userProfile'); Видаляє дані профілю з пам'яті браузера. Тепер сайт "забуде", хто був авторизований.

location.reload(); Примусово оновлює (перезавантажує) сторінку, щоб усі елементи інтерфейсу (кнопки "Увійти/Реєстрація") повернулися до стану як для неавторизованого гостя.

4. Додавання товару до кошика

```
//4. Додавання товару до кошика
function addToCart(productName, price) {
  cart.push({ name: productName, price: price });
  localStorage.setItem('cart', JSON.stringify(cart));
  showAlert(productName);
  updateCart();
}
```

function addToCart(productName, price): Функція, яка приймає два параметри: назву товару та його ціну.

cart.push({ ... }): Створює новий об'єкт з назвою і ціною та додає (пушить) його в кінець масиву cart.

localStorage.setItem('cart', JSON.stringify(cart)): Бере оновлений масив cart, перетворює його на текстовий формат (**JSON.stringify**) і перезаписує в пам'ять браузера під назвою 'cart'.

showAlert(productName): Викликає функцію для показу спливаючого повідомлення з назвою доданого товару.

updateCart(): Викликає функцію для оновлення відображення кошика на екрані.

5. Спливаюче сповіщення

```
//5. Спливаюче сповіщення
function showAlert(name) {
  const alertBox = document.getElementById('cart-alert');
  if (alertBox) {
    alertBox.innerText = `${name} додано до кошика!`;
    alertBox.style.display = 'block';
    setTimeout(() => alertBox.style.display = 'none', 3000);
  }
}
```

const alertBox = ...: Шукає HTML-елемент з **id="cart-alert"**.

if (alertBox) {: Перевіряє, чи такий елемент взагалі є на поточній сторінці (щоб уникнути помилки, якщо його немає).

alertBox.innerText = ...: Записує всередину блоку текст: "[Назва товару] додано до кошика!".

alertBox.style.display = 'block';: Робить цей блок видимим на сторінці (змінює CSS-стиль).

setTimeout(..., 3000);: Запускає таймер. Через 3000 мілісекунд (3 секунди) виконається команда **alertBox.style.display = 'none'**, яка знову сховає цей блок.

6. Оновлення списку товарів у кошику (на сторінці)

```
//6. Оновлення списку товарів у кошику (на сторінці)
function updateCart() {
  const list = document.getElementById('cart-items');
  const total = document.getElementById('total-price');
  if (!list || !total) return;
  list.innerHTML = '';
  let sum = 0;
  cart.forEach(item => {
    const li = document.createElement('li');
    li.textContent = `${item.name} - €${item.price}`;
    list.appendChild(li);
    sum += item.price;
  });
  total.innerText = sum;
}
```

const list = ... та **const total = ...**: Шукають на сторінці список (`<ul>` або `<ol>`) для товарів і блок для виводу загальної суми.

if (!list || !total) return;: Перериває виконання функції, якщо цих елементів немає на поточній сторінці (наприклад, ти на Головній, а кошик відображається тільки в Магазині).

list.innerHTML = ''; Повністю очищає список товарів на екрані, щоб заповнити його наново і не дублювати попередні записи.

let sum = 0; Створює змінну для підрахунку загальної суми, починаючи з нуля.

cart.forEach(item => {: Запускає цикл, який проходить по кожному товару (**item**) у масиві **cart**.

1.**const li = document.createElement('li')**; Створює новий HTML-тег `<li>` (елемент списку).

2.**li.textContent = ...**: Записує в цей тег текст: "Назва — €Ціна".

3.list.appendChild(li);: Додає цей готовий **** на сторінку в наш список list.

4.sum += item.price; Додає ціну поточного товару до загальної суми sum.

total.innerText = sum; Після завершення циклу виводить фінальну суму на сторінку.

7. Оформлення замовлення

```
//7. Оформлення замовлення
function checkout() {
  if (!user || !user.username) {
    alert("Щоб оформити замовлення, спочатку зареєструйтесь або увійдіть у профіль.");
    window.location.href = 'reg.html';
    return;
  }
}
```

if (!user...): Перевіряє, чи авторизований користувач.

alert(...): Якщо ні — показує системне спливаюче вікно з попередженням.

window.location.href = 'reg.html';: Одразу перенаправляє користувача на сторінку реєстрації.

return;: Зупиняє виконання функції (наступні рядки не виконуються).

8.Перевіряє, чи є щось у **масиві cart**. Якщо довжина **масиву 0** — видає попередження і зупиняє **функцію (return)**.

```
//8.Перевіряє, чи є щось у масиві cart. Якщо довжина масиву 0 – видає попередження і зупиняє функцію (return).  
if (cart.length === 0) {  
    alert("Ваш кошик порожній!");  
    return;  
}
```

9.Резюме: Цей код перетворює дані з масиву на читабельний чек, показує його клієнту, а потім "забуває" старі товари, готуючи сайт до нового замовлення.

```
let orderSummary = cart.map(item => ` - ${item.name}: €${item.price}`).join('\n');  
orderSummary += `\n\nЗагальна сума: €${cart.reduce((sum, item) => sum + item.price, 0)}$`;   
alert("Дякуємо за замовлення!\n\n" + orderSummary);  
cart = [];  
localStorage.removeItem('cart');  
updateCart();  
}
```

cart.map(...).join('\n'): Проходить по всіх товарах, формує для кожного рядок вигляду " - Назва: €Ціна", а потім склеює (join) усі ці рядки через символ переходу на новий рядок (\n). Зберігає це у змінну orderSummary (чек замовлення).

orderSummary += ...: Додає до цього чека текст "Загальна сума:" та вираховує загальну вартість за допомогою методу reduce (який складає ціни всіх товарів разом).

alert(...): Виводить на екран фінальне системне повідомлення: подяку та згенерований чек.

cart = []: Робить масив кошика в оперативній пам'яті порожнім.

localStorage.removeItem('cart'): Видаляє дані про кошик із пам'яті браузера, щоб після оновлення сторінки кошик знову не став повним.

updateCart(): Викликає функцію (яка, ймовірно, описана десь вище), щоб оновити інтерфейс сайту

(наприклад, змінити цифру кількості товарів у шапці сайту на "0").

10. Автоматичні дії при завантаженні сторінки

```
//10. Автоматичні дії при завантаженні сторінки
document.addEventListener('DOMContentLoaded', updateCart);
```

Чекає подію **DOMContentLoaded** (це момент, коли HTML-сторінка повністю завантажилась) і одразу запускає **функцію updateCart()**. Це робиться для того, щоб при оновленні сторінки кошик одразу відобразив збережені товари, а не був порожнім.

11. Це ще один обробник завантаження сторінки. Він потрібен для меню навігації.

```
//11. Це ще один обробник завантаження сторінки. Він потрібен для меню навігації
document.addEventListener('DOMContentLoaded', () => {
  const user = JSON.parse(localStorage.getItem('userProfile'));
  const authLinks = document.getElementById('auth-links');

  if (user && user.username) {
    authLinks.innerHTML = `👤 ${user.username} <button onclick="logoutUser()" style="margin-left: 1rem;">Вийти</button>`;
  }
});
```

Знову дістає користувача з **localStorage** і шукає блок навігації **auth-links** (де знаходяться посилання "Увійти" / "Реєстрація").

if (user...): Якщо користувач увійшов, він замінює посилання "Увійти/Реєстрація" на іконку профілю, ім'я користувача та кнопку "Вийти".

12. Дублікат функції

```
//12. Дублікат функції  
function logoutUser() {  
  localStorage.removeItem('userProfile');  
  location.reload();  
}
```

Це повна копія функції з блоку 3. Вона написана двічі. Програма працює, бо друга функція просто перезаписує першу (вони ідентичні), але в майбутньому цей шматок можна просто видалити, щоб код був чистішим.